# Sparse representation of images using gaussian splatting

**Victor Klomp**

November 21, 2025

**Abstract**

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri.

## 1 Introduction

In the last few years, AI and machine learning has become increasingly important. We have seen AI across a few domains, mainly text, image & video and audio. Each of these AI models work by using tokens as inputs for transformers. For text, these tokens are words or letters, however for images & video, defining these tokens is a bit more arbitrary. The most common approach is by taking 16x16 values of pixels, together with

## 2 Method

We will adapt the 2D gaussian splatting to include extra secondary data. This is the geometric concepts of the image, such as the rotation, scaling and translation. And we will try to incorporate higher frequency data of the image by using derivatives of the gaussian to be better able to plot edges and lines.

We will spawn $n$ initial gaussians, which will represent one token, and each token will have $p$ (tbd) parameters that describes the color values of the gaussian layers and the location, rotation and scale.

### 2.1 Gaussian layers

Instead of using just the gaussian function as is usual in gaussian splatting, we will also use the first and second order derivative of the gaussian function. As defined as

$$\phi_0(x) := e^{-\|x\|}$$

$$\hat{\phi}_1 = \frac{\mathrm{d}}{\mathrm{d}x_1}\phi_0$$

$$\hat{\phi}_2 = \frac{\mathrm{d}}{\mathrm{d}x_1}\phi_1$$

Please note that the chose of variable ($x_1$ or $x_2$) for the derivative does not matter, as the axis are orthogonal and the function can be rotated by $90\,\mathrm{deg}$ to get the other derivative.

This will give us more complex features to work with. These three gaussians will be layered in the same location. Below we can see the derivatives when they are unscaled.

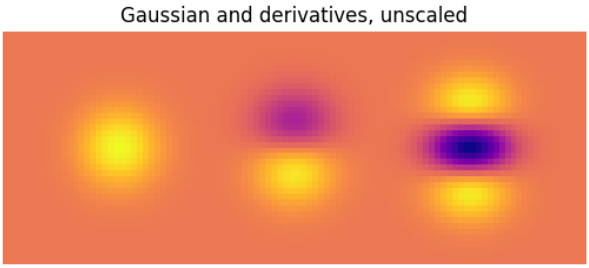


Figure 1: Unscaled: gaussian ($\phi_0$), first-order derivative ($\hat{\phi}_1$) and second-order derivative ($\hat{\phi}_2$) of a gaussian.

The idea is that the "zeroth-order" derivative is used as the base color, and that the derivatives are used as accents. The first-order derivative contains a fast drop-off, this could be used for creating a sharp edge, and the second order derivative contains a sharp bump in the middle this could be used as lines.

Layering these as is will not result in a good overlap however, because the support of these functions differs by a lot. We can scale the derivatives such that their support matches better with gaussian. Scaling the first-order by 0.7 and the second-order by 0.5, we get the following gaussians, which will be layered to create more complex structures.

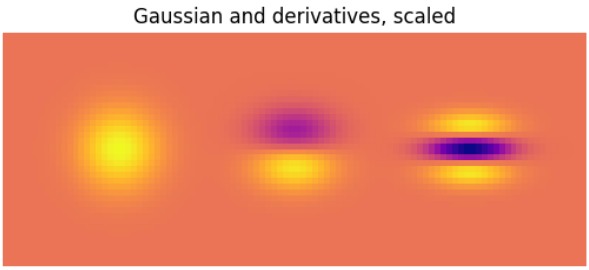


Figure 2: Scaled: gaussian ($\phi_0$), first-order derivative ($\phi_1$) and second-order derivative ($\phi_2$) of a gaussian.

Each of these gaussians have an independent color values. $c^{i,j}$, $i$ for each the gaussian layers, and $j$ for the color channels, totalling 9 values. Each color value is not directly the color that the gaussian attains, but rather a addition or substraction in a certain direction for the final color. For each pixel color $pc \in [0,1]^3$, we start with a default of 0.5. This has the advantage that the image is color invertible by multiplying all the color values by $-1$, but also does not introduce an implicit bias to the color black, as opposed to white (or vice versa). Therefor the color values of the gaussians can attain any value in $\mathbb{R}$, however, most likely they will be pushed between $[-0.5, 0.5]^3$ as the input image contains only values between $[0,1]^3$. This will be elaborated on later.

## 2.2 Lie groups

This component for position, rotation and scale should ideally be a lie group. As this means our representation has geometric properties.

### 2.2.1 Affine group

Such a group $G$ is defined as the following.

**Definition 2.2.1 (The affine group with positive determinants)** The *2-dimensional, real affine group with positive determinants* is defined as

$$G = \mathrm{Aff}^+(\mathbb{R}^2) := \mathbb{R}^2 \rtimes \mathrm{GL}^+(\mathbb{R}^2)$$

with the group operation given by

$$(x_2, A_2)(x_1, A_1) := (A_2 x_1 + x_2, A_2 A_1)$$

for all $(x_2, A_2), (x_1, A_1) \in \mathrm{Aff}^+(\mathbb{R}^2)$.

Where $\mathrm{GL}^+(\mathbb{R}^2)$ is the group of 2-dimensional real linear transformations with positive determinants, i.e. $2 \times 2$ matrices with positive determinant. This positive determinants ensures that the transformation does not flip the image, as the gaussians and their derivatives are symmetric, this is not needed, and only confuses.

**Todo**

What are the unit element and the inverse?

What is the dimension of the group?

Prove $\mathrm{Aff}^+(\mathbb{R}^2)$ is a *Lie group*.

The affine group acts on $\mathbb{R}^2$ in the following manner:

$$g \triangleright y = (x, A) \triangleright y := Ay + x$$

for all $y \in \mathbb{R}^2$ and $g = (x, A) \in \mathrm{Aff}^+(\mathbb{R}^2)$. Similarly for subsets $S \subset \mathbb{R}^2$ we define

$$g \triangleright S := \{g \triangleright y \mid y \in S\}.$$

This allows us to use this lie group to position points in our image. The next thing to consider is that the gaussians we use to fill our image is not one point, but a collection of points defined by a function in $\mathbb{R}^2$. Therefore we will also need to define how this group acts on a function on $\mathbb{R}^2$. This is induced as the following:

Let $f : \mathbb{R}^2 \to \mathbb{R}$ then we define $g \triangleright f : \mathbb{R}^2 \to \mathbb{R}$ by

$$(g \triangleright f)(y) := f(g^{-1} \triangleright y).$$

### 2.2.2 Lie groups & parameter space

This works quite well for positioning the gaussians in some particular spot. However, to make this match our target image, we will use gradient descent, as will be explained later. This requires the parameters to be a vector space. This is not the case by default for the Lie group. This means that when applying gradient descent at the current stage, the parameters could no longer be in the Lie group after gradient descent steps. This means that this parameter has no longer any useful meaning, and cannot be used to position the gaussian.

For this reason, we will search for a parameter space, $V$, such that $V$ is a vector space, and there exists a surjective function $\psi : V \to G$. We can use this to apply gradient descent on $V$, as this is a vector space, and render the image using $G$.

### 2.2.3 The Lie algebra $\mathfrak{aff}(\mathbb{R}^2)$

To solve this issue, we will make use of the Lie algebra. This is the tangent space at the identity of the Lie group. The Lie algebra can map to the Lie group (and vice versa), but is also a vector space.

The Lie algebra of $\mathrm{Aff}^+(\mathbb{R}^2)$ is $\mathfrak{aff}(\mathbb{R}^2) := \mathbb{R}^2 \rtimes \mathfrak{gl}(\mathbb{R}^2) \equiv \mathbb{R}^2 \rtimes \mathbb{R}^{2\times 2}$. Note that we say $\mathfrak{aff}$ and not $\mathfrak{aff}^+$, this is because $\mathfrak{aff}$ is the Lie algebra of both the $\mathrm{Aff}^+$ and Aff Lie groups, illustrating that there is not a one-to-one relationship between Lie groups and Lie algebras.

The lie algebra $\mathfrak{aff}$ can be represented by the linear transformation $W$ and a translation $v$ seperately. Or they can be put together in one matrix, as

$$A = \begin{pmatrix} W & v \\ 0 & 0 \end{pmatrix}$$

### 2.2.4 The exponential & logarithmic map

Lie groups and Lie algebras are related to each other through the exponential map. For our case the exponential map $\exp_{\mathrm{Aff}^+(\mathbb{R}^2)} : \mathfrak{aff}(\mathbb{R}^2) \to \mathrm{Aff}^+(\mathbb{R}^2)$ is given by $\exp(A)$.

Doing this for $W$ and $v$ seperately is equivalent to

$$\exp((v, W)) := \left( \left( \int_0^1 e^{tW} \, \mathrm{d}t \right) v, e^W \right). \tag{1}$$

Unfortunately, from (Culver 1966) we know that matrix multiplication is not surjective. So we cannot use $\mathfrak{aff}$ to create every possible gaussian. Besides the fact that is not surjective, is the integral also a problem. This means that the function is not a convenient close formula, but instead requires a lot of computation.

## 2.3 Kaji-Ochiai parameterization

As we cannot use the default Lie algebra $\mathfrak{aff}$ from the Lie group $\mathrm{Aff}^+$ as $V$, we will need to look to a bit further. We adapt the findings from (Kaji and Ochiai 2016), where they describe a parameterization for $\mathrm{Aff}^+(\mathbb{R}^3)$, which has a similiar roles as the Lie algebra. This parameterization is for 3D affine transformation, it starts from what is essentially the Lie algebra but constructs a different surjective mapping than the exponential onto the group (but still closely related). We will simplify this parametrization to 2 dimensions.

The Kaij-Ochiai parameterization works by factoring the Lie algebra of $\mathrm{Aff}^+(\mathbb{R}^2)$ as per

$$\mathfrak{aff}(\mathbb{R}^2) \equiv \mathbb{R}^2 \times \mathfrak{so}(2) \times \mathfrak{sym}(2) \equiv \mathbb{R}^2 \times \mathbb{R} \times \mathbb{R}^3 \equiv \mathbb{R}^6$$

where $\mathbb{R}^2$ encodes the translation generator, $\mathfrak{so}(2)$ the rotation generator and $\mathfrak{sym}(2)$ the scale and shearing generator. Here $\mathfrak{so}(2)$ is the Lie algebra of $\mathrm{SO}(2)$ and is given by the real $2 \times 2$ anti-symmetric matrices and so has only a single degree of freedom:

$$\mathfrak{so}(2) = \left\{ \begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix} \middle| \, r \in \mathbb{R} \right\}.$$

The second vector space $\mathfrak{sym}(2)$ is the set of (real) symmetric $2 \times 2$ matrices, but it is not the Lie algebra of any Lie group since the Lie bracket of two symmetric matrices is not necessarily a symmetric matrix, it is anti-symmetric in fact:

$$[A,B]^T = (AB - BA)^T = B^T A^T - A^T B^T = -(A^T B^T - B^T A^T) = -[A,B]$$

for all $A, B \in \mathfrak{sym}(n)$.

The vector space $\mathfrak{sym}(2)$ has three degrees of freedom:

$$\mathfrak{sym}(2) = \left\{ \begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix} \;\middle|\; s_1, s_2, s_3 \in \mathbb{R} \right\}.$$

**Definition 2.3.1** The parametrization map $\phi : \mathbb{R}^2 \times \mathbb{R} \times \mathbb{R}^3 \to \mathrm{Aff}^+(\mathbb{R}^2)$ is given by

$$\phi(x, r, s) := \left( x, \exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix} \exp\begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix} \right). \tag{2}$$

The matrix exponentials in the parametrization map 2 have the following closed formulae

$$\exp_{\mathfrak{so}} = \exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix} = \begin{pmatrix} \cos r & -\sin r \\ \sin r & \cos r \end{pmatrix}$$

and

$$\begin{aligned} \exp_{\mathfrak{sym}} = \exp\begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix} &= e^{\frac{s_1+s_2}{2}} \begin{pmatrix} \cosh(a) + \frac{s_1-s_2}{2a}\sinh(a) & \frac{s_3}{a}\sinh(a) \\ \frac{s_3}{a}\sinh(a) & \cosh(a) + \frac{s_2-s_1}{2a}\sinh(a) \end{pmatrix} \\ &= e^{\frac{s_1+s_2}{2}} \left( \cosh(a)\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \frac{\sinh(a)}{a} \begin{pmatrix} \frac{s_1-s_2}{2} & s_3 \\ s_3 & \frac{s_2-s_1}{2} \end{pmatrix} \right) \end{aligned}$$

where $a := \frac{1}{2}\sqrt{(s_1 - s_2)^2 + 4s_3^2}$.

Note that $\det\left(\exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix}\right) = 1$ and $\det\left(\exp\begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix}\right) = e^{s_1+s_2} > 0$.

Furthermore, we can also look at the inverse of the parameterization.

**Definition 2.3.2** A locally differentiable inverse $\overleftarrow{\phi} : \mathrm{Aff}^+(\mathbb{R}^2) \to \mathbb{R}^2 \times \mathfrak{so}(2) \times \mathfrak{sym}(2)$ is given by

$$\overleftarrow{\phi}(x, A) := \left( x, \log\left( A(A^T A)^{-\frac{1}{2}} \right), \log\left( \sqrt{A^T A} \right) \right). \tag{3}$$

This local inverse relies on the *polar decomposition* of a square matrix (Hall 2015, § 2.5). Every square matrix $A$ admits a polar decomposition into an orthogonal matrix $R$ and and a positive semi-definite matrix $S$ so that $A = RS$. If $A$ is invertible, which it is in our case, then the polar decomposition is unique and $S$ is positive definite. The matrices $R$ and $S$ are calculated as per

$$R = A(A^T A)^{-\frac{1}{2}} \quad \text{and} \quad S = (A^T A)^{\frac{1}{2}}.$$

Since $A$ is invertible $A^T A$ is symmetric positive definite and so has a unique square root which is itself symmetric positive definite and hence uniquely invertible. Clearly $A = RS$ and $S$ is symmetric positive definite, orthogonality of $R$ follows from

$$R^T R = \left( A(A^T A)^{-\frac{1}{2}} \right)^T \left( A(A^T A)^{-\frac{1}{2}} \right) = (A^T A)^{-\frac{1}{2}} (A^T A)(A^T A)^{-\frac{1}{2}} = I.$$

**Note**

Some formulae for computing an inverse (need to verify):

$$s_1 + s_2 = \log(\det(S)) = \log(\det(A)),$$
$$\cosh(a) = 2e^{-\frac{s_1+s_2}{2}} \operatorname{trace}(S),$$
$$s_1 - s_2 = e^{-\frac{s_1+s_2}{2}} \frac{a}{\sinh(a)}(S_{1,1} - S_{2,2}),$$
$$s_3 = e^{-\frac{s_1+s_2}{2}} \frac{a}{\sinh(a)} S_{2,1}.$$

## 2.4 A note on shearing

Shearing is the changing the angle of the orthogonal base vectors of the space. No shearing implies that $e_1^T e_2 = 0$. This shearing can have some interesting properties, so we will take a closer look to see if we want this or if we want to exclude this from the Gaussian transformations.

Disecting the polar decomposition from earlier, we can see that $R \in \mathrm{SO}(2)$ and thus does rotation, and $S \in \mathrm{SPD}(2)$ does scaling and this shearing.

We define $S$ as

$$S = \begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix}$$

.

Then we can see that $s_3$ generates shearing, as when $s_3 = 0$ then we have $(Se_1)^T(Se_2) = (s_1, 0)^T(0, s_2) = 0$. And vice versa, if $s_3$ is variable, then $Se_1)^T(Se_2) = (s_1, s_3)^T(s_3, s_2) = s_1 s_3 + s_3 s_2$, clearly introducing a shear.

Futhermore, we can see that the effect of a shear on a Gaussian can also be generated by a rotation and a scale. I.e. we can remove $s_3$ and still generate all Gaussian configurations.

*Proof:*

First, we know that any $S \in \mathrm{SPD}(2)$ can be generated by $R \in \mathrm{SO}(2)$ and $\hat{S} \in D$ where $D$ is all diagonal matrices. This is an eigendecomposition, defined as the following $S = R^T \hat{S} R$.

Then we also notice that rotation in the Stabilizer of $\phi_0$. I.e.

$$\mathrm{Stab}_G(\phi_0) := \{g \in G \mid g \triangleright \phi_0 = \phi_0\} \equiv \mathrm{SO}(2)$$

This means that we get the following

$$S \triangleright \phi_0 = R^T \hat{S} R \triangleright \phi_0 = R^T \hat{S} \triangleright (R \triangleright \phi_0) = R^T \hat{S} \triangleright \phi_0$$

Then we will show that first scaling and then rotating does not result in shearing.

We will take the basis of the space $e_1$ and $e_2$, these are orthogonal if their inner product is zero. We know the basis is orthogonal by definition, thus $e_1^T e_2 = 0$

We can see that any transformation $A = RS$ with

$$R = \begin{pmatrix} \cos r & -\sin r \\ \sin r & \cos r \end{pmatrix}$$

and

$$S = \begin{pmatrix} \exp(s1) & 0 \\ 0 & \exp(s2) \end{pmatrix}$$

applied as $RSx$ results in an orthogonal "space"(not sure what the correct word is here).

We can see this by

$$\begin{aligned} (Ae_1)^T(Ae_2) &= (RSe_1)^T(RSe_2) \\ &= e_1^T S^T R^T RSe_2 \\ &= e_1^T S^T Se_2 \qquad \text{As R is orthogonal} \\ &= (s_1, 0)^T(0, s_2) \\ &= 0 \end{aligned}$$

.

It is also true that applying $SRx$ in the other order can result in shearing, we will show this with an counter example.

Define

$$S = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$$

and

$$R = \begin{pmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{pmatrix}$$

.

Now clearly $(SRe_1)^T(SRe_2) = -\frac{3}{2} \neq 0$

■

And thus we can see that $\phi_0$ is not affected by shearing.

We can remove the shearing by setting $s3 = 0$ in the parameter space $V$. Furthermore, from the proof we can also see that we multiply the $\exp_{\mathfrak{so}}$ with $\exp_{\mathfrak{sym}}$ instead of the otherway around, i.e. applying the scaling before the rotation. This is currently already the case, in the parametrization of Kaij-Ochiai.

Now that we have fixed $s3 = 0$, we can see that $V \equiv \mathbb{R}^5$. We can still map this using $\psi$, however this will no longer map to $\mathrm{Aff}^+$, but a subset of that where there exists no shearing. This subset is not a group.

*Proof:*

Take $R$ and $S$, which are each clearly in this subset that contains no shearing. However as we showed before, $SR$ does contain shearing. Thus not in this subset, therefore the subset is not a group.

■

The first and second order Gaussians are not rotationally symmetric, so shear does add an unique property here.

However, emoving this property gives us some better analysis on the anisotropy[1] of a Gaussian, as the anisotropy is now uniquely generated by the scaling parameters. However we will need to do some experiments if the shear property has significant impact on the first and second derivative.

## 2.5 Rendering function

For $x \in \mathbb{R}^2$ the Gaussian value is defined as $\phi(x) = \exp(-\|x\|)$.

The target image we want to approximate is defined as $f(x) : \mathbb{R}^2 \to \mathbb{R}^3$.

The final image that is generated from the parameters is defined by

$$\hat{f}(x) = \sum_{k=0}^{n} \sum_{i}^{3} c_k^i (g_k \rhd \phi)(x) = \sum_{k=0}^{n} \sum_{i}^{3} c_k^i (\psi(v_k) \rhd \phi)(x)$$

To find the optimal parameters $c$ and $v$, we will use gradient descent as explained earlier. For this, we will define a score, called a loss, that determines how well a set of parameters represents the image. We will use two methods to measure how well a representation is. First of all, we will use the $L_1$ norm, in practice, this is the pixel-by-pixel difference of each color channel. Formally defined as $\|f - \hat{f}\|$. Besides this we will also use the structural similarity index measure (SSIM). These will be weighed with $\lambda$. The total loss for how well the image looks will be $L_{\text{image}} = \lambda\|f - \hat{f}\| + (1 - \lambda) * \left(1 - \text{SSIM}\left(f, \hat{f}\right)\right)$.

Besides the actual representation, we have two other things that we want to discourage. As gradient descent tries to find the minimum of the loss, we can put things in the loss to discourage certain behavior. First of all, we want to limit the anisotropy, to make sure gaussians do not become stretched out, but stay fairly round. This will prevent artifacts and overfitting. Futhermore, we also do not want gaussians to become too small, to be exact they must not be smaller than a pixel, as this will make them hidden while rendering, but they still contain information in the final embedding.

We will add two extra components to the loss function, one for the sizing and one for the anisotropy. They are defined as $L_{\text{anisotropy}} = \overline{|s_1 + s_2|} * \lambda_{\text{anisotropy}}$ and $L_{\text{sizing}} = \overline{\exp(-(s_1 + s_2) - 8)} * \lambda_{\text{sizing}}$. Where $\lambda_{\text{anisotropy}}$ and $\lambda_{\text{sizing}}$ are how much influence these loss functions have.

The final loss is then defined as $L = L_{\text{image}} + L_{\text{anisotropy}} + L_{\text{sizing}}$

As $\mathbb{R}^2 \times \mathbb{R} \times \mathbb{R}^3$ are all vector spaces and the loss function is continuous we can use gradient descent to optimize this. We will use AdamW and a scheduler to futher optimize the trainig process.

## 2.6 Culling

As we want to decrease the amount of gaussians that we save, we will remove the gaussians based on their parameters, this process we will call culling. The most obvious is that we would like to remove gaussians where the color values are negligible small for all gaussian layers. Formalized by

[1] Anisotropy is how "stretched" an ellipsoid, or in our case a Gaussian, is. We will see later why this is useful

$$\sum_{j}^{3} c^{1,j} \leq 0.05 \wedge \sum_{j}^{3} c^{2,j} \leq 0.05 \wedge \sum_{j}^{3} c^{3,j} \leq 0.05$$

.

The other criterion is again the anisotropy of the gaussian, as mentioned earlier. Formally, we do this by removing all gaussians where

$$s1 + s2 \leq 7.5$$

.

The problem is that these gaussians are visible, so doing this culling after all training is done, would leave some gaps in the image. Therefore we do this on 80% of the training, thus giving the gradient descent still some iterations to recover and fill the gaps.

# 3 Results

To put this into practise, we use pytorch to make use of CUDA kernels and speed up the rendering. The source code is available at (). The images are constrained to 1:1 to ease the implementation, the theory can be easily scaled to any ratio.The resolution can differ, and based on the input. For the experiments, mostly 100x100 resolutions are used, as this is a limitation of the physical hardware where the tests were run on.

The hardware used is an `Intel(R) Core(TM) i7-10750H (12) @ z` and `NVIDIA Quadro T1000 Mobile`, on this hardware training took between 1 minute and 5 minutes. However, this is not a goal of the project.

In this section we will discuss the results and adaptations we have made based on the findings. We will first present the final product which includes all the adaptations, which we will use as our base performance to compare against.

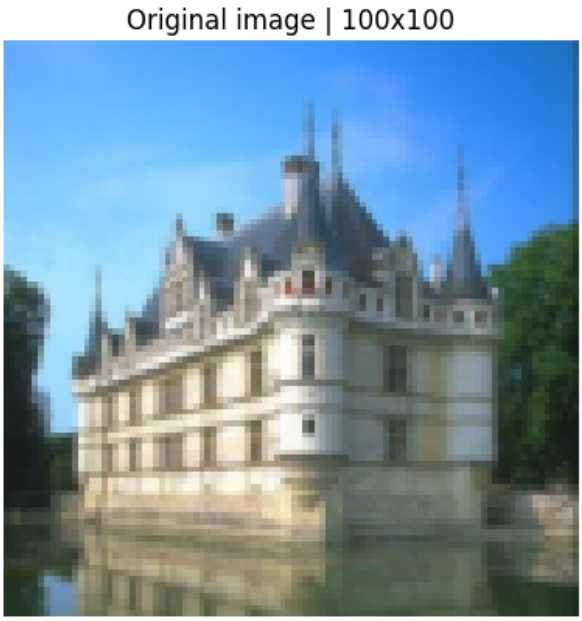


Figure 3: Original castle image

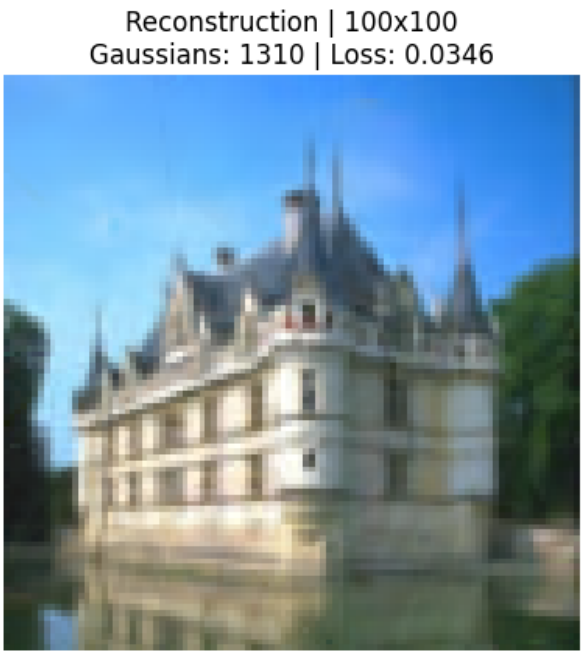


Figure 4: Base performance of the representation

### 3.1 Initalization

For the initalization we use an hyperparameter for the amount of Gaussians that will be spawned. The color parameters of $\phi_0$ will be initialized with a normal distribution $N(0,1)$, for $\phi_1$ and $\phi_2$ this is $N(0,0.1)$. This prevents the first and second order from being to active. The position of the Gaussians is also initialized with $N(0,1)$. The scaling parameters are also generated with $N(0,1)$ but immediately shifted down with $-4$. If this is not done, the Gaussians will overlap too much, and fight against each other, resulting in streaks. This can be seen below.

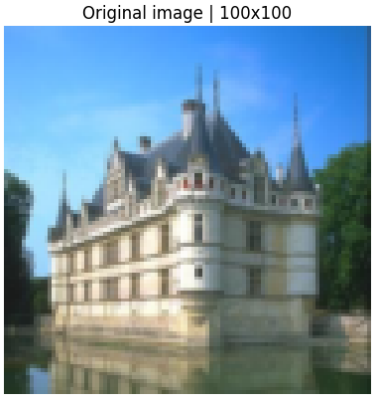


Figure 5: Original castle image

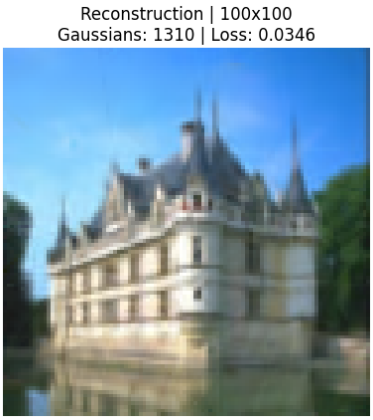


Figure 6: Base performance of the representation

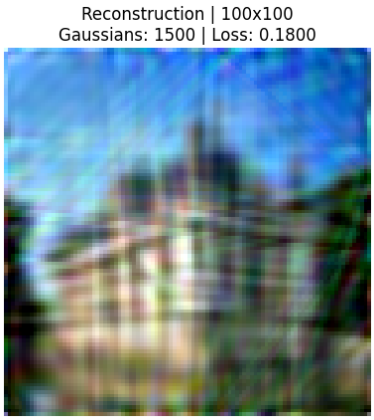


Figure 7: Representation with unscaled initalization

## 4 Appendix section

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

## Bibliography

Chen G, Wang W (2024) A Survey on 3D Gaussian Splatting. http://arxiv.org/abs/2401.03890. Accessed 18 Dec 2024

Cohen T, Welling M (2016) Group equivariant convolutional networks. In: International conference on machine learning. pp 2990–2999

Culver WJ (1966) On the existence and uniqueness of the real logarithm of a matrix. Proceedings of the American Mathematical Society 17:1146–1151

Daugman J (1988) Complete discrete 2-D Gabor transforms by neural networks for image analysis and compression. IEEE Transactions on Acoustics, Speech, and Signal Processing 36:1169–1179. https://doi.org/10.1109/29.1644

Dong J, Wang C, Zheng W, et al (2025) GaussianToken: An Effective Image Tokenizer with 2D Gaussian Splatting. http://arxiv.org/abs/2501.15619. Accessed 10 July 2025

Dosovitskiy A, Beyer L, Kolesnikov A, et al (2021) An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. http://arxiv.org/abs/2010.11929. Accessed 12 Jan 2023

Hall BC (2015) Lie Groups, Lie Algebras, and Representations: An Elementary Introduction. Springer International Publishing, Cham

Hu J, Xia B, Chen B, et al (2024) GaussianSR: High Fidelity 2D Gaussian Splatting for Arbitrary-Scale Image Super-Resolution. https://ui.adsabs.harvard.edu/abs/2024arXiv240718046H. Accessed 10 July 2025

Kaji S, Ochiai H (2016) A Concise Parametrization of Affine Transformation. SIAM Journal on Imaging Sciences 9:1355–1373. https://doi.org/10.1137/16M1056936

Kerbl B, Kopanas G, Leimkühler T, Drettakis G (2023) 3d gaussian splatting for real-time radiance field rendering.. ACM Trans Graph 42:139–131

Martin D, Fowlkes C, Tal D, Malik J (2001) A database of human segmented natural images and its application to evaluating segmentation algorithms and measuring ecological statistics. In: Proceedings eighth IEEE international conference on computer vision. ICCV 2001. pp 416–423

Niemeijer M, Staal J, Van Ginneken B, et al (2004) Comparative study of retinal vessel segmentation methods on a new publicly available database. In: Medical imaging 2004: image processing. pp 648–656

Peng L, Wu A, Li W, et al (2025) Pixel to Gaussian: Ultra-Fast Continuous Super-Resolution with 2D Gaussian Modeling. https://ui.adsabs.harvard.edu/abs/2025arXiv250306617P. Accessed 10 July 2025

Smets BM, Portegies J, Bekkers EJ, Duits R (2023) PDE-based group equivariant convolutional neural networks. Journal of Mathematical Imaging and Vision 65:209–239

Tai Sing Lee (1996) Image representation using 2D Gabor wavelets. IEEE Transactions on Pattern Analysis and Machine Intelligence 18:959–971. https://doi.org/10.1109/34.541406

Zhang X, Ge X, Xu T, et al (2025a) GaussianImage: 1000 FPS Image Representation and Compression by 2D Gaussian Splatting. In: Leonardis A, Ricci E, Roth S, et al. (eds) Computer Vision – ECCV 2024. Springer Nature Switzerland, Cham, pp 327–345

Zhang Y, Li B, Kuznetsov A, et al (2025b) Image-gs: Content-adaptive image representation via 2d gaussians. In: Proceedings of the Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers. pp 1–11

Zhu L, Lin G, Chen J, et al (2025) Large Images are Gaussians: High-Quality Large Image Representation with Levels of 2D Gaussian Splatting. http://arxiv.org/abs/2502.09039. Accessed 10 July 2025